Concept Exploration for LLMs and Diffusion Models

Sparse Auto Encoders for Mechanistic Interpretability

[1]**Département de mathématiques** - *Université du Québec à Montréal*
[2]**Institut Intelligence et Données** - *Université Laval*

Bootcamp: Statistical Insights into Modern AI Systems
IVADO Thematic Semester - Statistical Foundations of AI
May 7, 2026 - Montréal QC, Canada

Marouane IL Idrissi[12]

Agenda

Day 4 - May 4, 2026

9:30 AM – 11:00 AM – Lecture 1: Feature importance and Cooperative Games

11:00 AM – 11:30 AM – Coffee Break

11:30 AM – 12:15 PM – Lab 1: Cooperative Games from scratch (Python)

12:15 PM – 2:00 PM – Lunch

2:00 PM – 2:45 PM – Lab 2: Showcase of various use-cases (Python & R)

2:45 PM – 3:30 PM – Lecture 2: Exploring concepts in LLMs and Diffusion models ← We are here

3:30 PM – 4:00 PM – Coffee Break

4:00 PM – 4:45 PM – Lab 3: SAEs for LLMs and Diffusion Models

Notes

Outline

1. Mechanistic interpretability
2. Exploring concepts in LLMs
3. Exploring concepts in Stable Diffusion models
4. Practical use-cases of unpacked concepts

Outline

1. Mechanistic interpretability
2. Exploring concepts in LLMs
3. Exploring concepts in Stable Diffusion models
4. Practical use-cases of unpacked concepts

Notes

Mechanistic interpretability

Mechanistic interpretability aims to ***understand the computational mechanisms*** *underlying* ***neural networks'*** *capabilities to accomplish* ***concrete scientific and engineering goals****.*
- Sharkey et al. (2025)

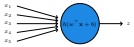
Several goals (non-exhaustive):

- AI monitoring to avoid dangerous behavior
- Modify the models' internals mechanisms to adapt their behavior
- Predict the models' behavior on unseen situations
- Improve model inference
- **Extract latent knowledge from AI systems**

☞ **This part of the lecture: primarily the ML mindset**

Multilayer perceptron

Multilayer perceptrons (MLP) are a particular **class of models**
☞ a.k.a. feed-forward neural networks



Neurons:

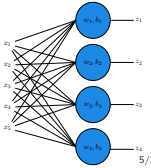
- Inputs: $\mathbf{x} \in \mathbb{R}^d$
- Activation function: h differentiable **(hyper-parameter)**
- Parameters: $\boldsymbol{w} \in \mathbb{R}^d$, $b \in \mathbb{R}$

☞ **Differentiable parametric function** $\mathbb{R}^d \to \mathbb{R}$

Layer:

- Stack neurons to define a multivariate-valued function
- **Number of neurons** m is a **hyper-parameter**

☞ **Differentiable parametric function** $\mathbb{R}^d \to \mathbb{R}^m$



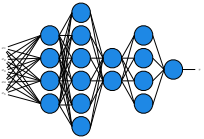
Notes

Multilayer perceptron

Multilayer perceptron:

- **Composition** of different layers
- **Number of layers** q is a **hyper-parameter**

☞ **Differentiable parametric function** $\nu_{\mathbf{w},\mathbf{b}} : \mathbb{R}^d \to \mathbb{R}$



With a n-sample dataset $(\mathbf{x}, \mathbf{y})$ and a **loss function** $\ell(x, y)$ we seek

$$(\mathbf{w}^*, \mathbf{b}^*) \in \underset{w,b}{\operatorname{argmin}} \; R(\mathbf{w}, \mathbf{b}) = \sum_{i=1}^{n} \ell(\nu_{\mathbf{w},\mathbf{b}}(\mathbf{x}^i), y^i)$$

☞ $R(\mathbf{w}, \mathbf{b})$ **is (generally) not strictly convex and** $\nu_{\mathbf{w},\mathbf{b}}$ **is over-parametrized...**
☞ **We can still compute its gradient**, and **perform gradient-descent type optimization**

Feels illegal, but it performs exceptionally well in practice
(ML mindset)

Exploiting representations of a multilayer perceptron

Mechanistic interpretability is convinced that:

- An MLP's layers **infers concepts** (of features) of varying level
  ☞ Stripes, ears, giraffe, golden gate bridge...
- It is **possible to recover these concepts from a trained neural network**
  ☞ But it is not easy...

☞ For a layer valued in $\mathbb{R}^m$, a concept **is a vector (direction) in** $\mathbb{R}^m$
☞ Two concepts are similar if their **inner product** high
☞ Two concepts are dissimilar if they are **orthogonal**

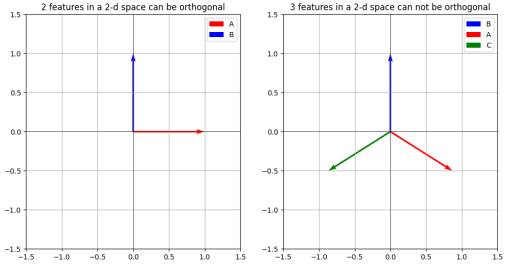
Problem: There is (usually) more than m different concepts
And only m orthogonal vectors can span $\mathbb{R}^m$...

Superposition hypothesis

Neural networks want to represent more concepts than they have neurons, so they exploit the properties of high-dimensional spaces - Olah et al. (2020)

Notes

Superposition hypothesis



Superposition hypothesis

In fact, one can embbed up to $\exp(m)$ "**almost orthogonal**," vectors in $\mathbb{R}^m$

☞ This is the Johnson–Lindenstrauss lemma.

Consequence: Concepts are not represented by **single neurons**

☞ The concepts can activate several neurons

This is why neural networks a inherently hard to interpret!

But also probably why these models scale so well for complex tasks

This phenomenon has been **observed and validated empirically**

Notes

In a nutshell

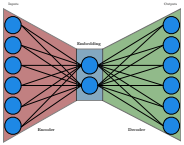
Let's recap:

- Neural networks are a composition of layers
- These layers take vectors as inputs and returns vectors as outputs
- Concepts are vectors in the output space ($\mathbb{R}^m$)
- For complex tasks (image, text), we need more than m concepts
- Superposition hypothesis: MLPs cram more concepts in the same space
- This explains why MLPs scale so well
- And why scrutinizing neurons directly yield little to no insights

Is there a way to extract a layer's concepts?

Traditional autoencoders

Traditional autoencoders **embed** a layer's output into **a lower-dimensional space**



It is an MLP trained to **reconstruct a layer's output**:

- Pass data into an MLP, cut at a specific layer and gather a bunch of vectors
- The inputs = the outputs = these vectors
- The bottleneck **forces dimension reduction**

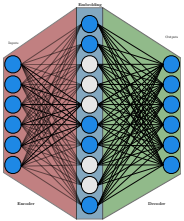
Very useful for denoising, anomaly detection, dimension reduction...

☞ **They do the opposite of what we want...**

Notes

Sparse autoencoders

Sparse autoencoders (SAE) **embed** a layer's output into **a higher-dimensional space**



It is an MLP trained to **reconstruct a layer's output**:

- Pass data into an MLP, cut at a specific layer and gather a bunch of vectors
- The inputs = the outputs = these vectors
- The middle layer **forces dimension expansion**
- **Sparsity constraints** through **the empirical risk**

Sparsity **forces the middle layer to represent concepts**

☞ Empirical behavior (again, ML mindset)

☞ **SAEs "unpack" superposed concepts**

Discussion

SAEs **are not reliable**:

- Unpacked concepts are not guaranteed to make sense
- Hard to interpret (because they're MLPs)
- Sensitive to training data and initialization (because they're MLPs)
- Sensitive to their structure (number of neurons, width...)
- "Use a black box to understand a black box"

But,

☞ They are a **tangible solution to a complicated problem**

☞ They showed **promising empirical results** for a bunch of models

(Again, ML mindset)

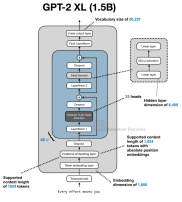
Notes

Outline

1. Mechanistic interpretability
2. Exploring concepts in LLMs
3. Exploring concepts in Stable Diffusion models
4. Practical use-cases of unpacked concepts

What is an LLM?

LLMs are a composition of an **attention block** and an **MLP** repeated several times



Attention blocks inject **context** to a word embedding
☞ They output a modified word embedding

MLPs are responsible for **activating concepts**
☞ They are the brains of the whole operation

Research in mechanistic interpretability for LLMs:

- Probe different MLPs in an LLM
- Train SAEs on retrieved activations
- Employ strategies to interpret learned concepts

Credits sebastianraschka.com

Rule of thumb: The later the MLP, the higher-level the concepts

Notes

How to train an SAE for an LLM

Start with a **diverse set of prompts**, ranging a **lot of topics**

Each word (token) of the prompt is represented as a vector in $\mathbb{R}^n$
☞ Bunch of vectors in $\mathbb{R}^n$ ($n = 12,288$ for GPT-3)

Probe one of the LLM's layer. Feed each token to the LLM and **save the layer's activations**
☞ **Dataset of activations:** rows are the tokens of the prompts, and $\mathbb{R}^m$ columns ($m = 4 \times n$ for GPT-3)

Train an SAE to **reconstruct the dataset**
☞ An MLP from $\mathbb{R}^m$ to $\mathbb{R}^m$

Several flavors of SAEs to chose from

☞ Pick "the best one" (ML mindset)

Making sense of unpacked concepts for LLMs

Start with another **diverse set of prompts**, ranging a **lot of topics**

Each word (token) of the prompt is represented as a vector in $\mathbb{R}^n$
☞ Bunch of vectors in $\mathbb{R}^n$ ($n = 12,288$ for GPT-3)

Probe one of the LLM's layer. Feed each token to the LLM and **save the layer's activations**
☞ **Dataset of activations:** rows are the tokens of the prompts, and $\mathbb{R}^m$ columns ($m = 4 \times n$ for GPT-3)

☞ **Each activation in $\mathbb{R}^m$ is linked to a word (token)**

Feed each activation to the **trained SAE's <u>encoder</u>**
☞ You get a bunch of SAE activations

☞ For every SAE neurons, retrieve the top-k words (tokens) that **activated it the most**
And hope it gives out something meaningful

(again, ML mindset)

Notes

In a nutshell

Using SAEs for LLMs:

- Pick an LLM, and one of its MLPs
- Gather a bunch of diverse prompts
- Cut the MLP at a specific layer, and train an SAE to unpack this layer
- Gather another bunch of diverse prompts
- Retrieve the MLP layer's activation, pass them to the SAE
- Record which SAE neuron activated from which word
- Retrieve the words that activated each SAE neuron the most
- Summarize these words around a general "concept"

☞ In some instances, the retrieved concepts can be meaningful
Québec or Statistics, from Neuronpedia

Outline

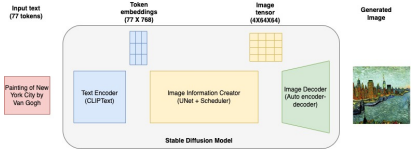
1. Mechanistic interpretability
2. Exploring concepts in LLMs
3. Exploring concepts in Stable Diffusion models
4. Practical use-cases of unpacked concepts

Notes

What is a Stable Diffusion Model?

The **Stable Diffusion Architecture** has three big components

1. **Text encoder:** Takes a prompt and encodes it into vectors
2. **Latent Denoiser:** Takes a noisy tensor and a prompt, and returns a "denoised" vector
3. **Variational autoencoder:** Takes a denoised tensor and returns a colored image



Making sense of Stable Diffusion

The idea behind Stable Diffusion is **to go back in time** by **denoising a diffusion processess**

Gaussian random walk:

- Start with a random value $z_0 \sim \mathcal{N}(0,1)$
- Iterate $z_{i+1} = z_i + \epsilon_i, \epsilon \sim \mathcal{N}(0,1)$

With the knowledge of

- The law to ϵ_i, for every step $i = 1, \ldots, m$
- The total number of steps taken m
- The observed state at timestep z_m

It is possible to have an educated guess of what z_0 was (Ho et al. 2020)

It is possible to approximate z_{i-1} as a function of z_i and ϵ_i
☞ It is called **the scheduler**

Notes

How do we train a Stable Diffusion model?

☞ **Text Encoder:** Pretrained embedding (e.g., CLIP) (Radford et al. 2021)

☞ **(Variational) autoencoder:** Trained to reconstruct **captioned images**

☞ **Training the U-net:**

- Pass the images into the **trained encoder part of the autoencoder (cut at the bottleneck)**
  ☞ Encoded images (tensors) $z^1, \ldots, z^n$ are called **latents**
- Pass the **captions into the text encoder**
  ☞ Encoded captions (vectors) $c^1, \ldots, c^n$
- Iteratively degrade each latent by adding noise **and record the added noise**
  ☞ $z_0^1 = z^1, z_1^1 = z_0^1 + \epsilon_0^1 \ldots$
- Use the dataset to **predict the noise** using the **latent, timestep and caption**

Latent	Timestep	Caption	Noise
z_0^1	0	c_1	0
z_1^1	1	c_1	ϵ_1^1
$\vdots$	$\vdots$	$\vdots$	$\vdots$

☞ The U-Net's noise prediction **is conditional to the caption**

And this is why the magic happens

How to generate an image from a Stable Diffusion model?

General idea behind an image generation pipeline:

1. Write a prompt of what the image should be
2. Pass the prompt to the text encoder, and get back a vector c^*
3. Randomly generate a latent z_m^*
4. Pass (z_m^*, c^*, m) to the U-Net and get a prediction ϵ_m^*
5. Pass ϵ_m^* and z_m^* into the scheduler, and get z_{m-1}^*
6. Iterate until you get to z_0^*
7. Pass the "denoised" latent z_0^* to the **decoder part of the autoencoder**
8. The image has been generated

☞ **Works because the noise predictions are conditioned on the prompt**

(The actual pipeline is much more complicated with many more tricks and intricacies...)

Notes

Unpacking concepts from a Stable Diffusion model

Usually, sparse autoencoders are trained **on the latter layers of the U-Net**
☞ Usually leads to higher-level concepts

To make sense of concepts, we can use a **captioned images** (e.g., CoCo dataset):

1. Pass the image to the **encoder** of the autoencoder to get latents
2. Pass the captions to the text encoder to get text vectors
3. Give the latent, the text vector, and the timestep 0 to the U-Net
4. Record the activation of the selected layer, and pass them to the SAE
5. Check which SAE neurons are activated and record their captions
6. For each SAE neuron, look at the captions of the images that activated them the most
7. Regroup these captions around a general concept

Outline

1. Mechanistic interpretability
2. Exploring concepts in LLMs
3. Exploring concepts in Stable Diffusion models
4. Practical use-cases of unpacked concepts

Notes

Practical use-cases: an overview

Once concepts are unpacked, they can be **actively exploited**:

- **Activation steering:** amplify or suppress concepts at inference
- **Model auditing:** detect undesirable or biased concepts

☞ **In all cases: manipulate SAE neurons, not model weights directly**

Activation steering

Idea: intervene on a layer's activations **at inference time** to nudge the model's behavior

Procedure:

1. Identify the SAE neuron(s) encoding the concept of interest
   ☞ e.g., the "Banana" neuron, or a "formal tone" neuron
2. At inference, **increase** or **decrease** the SAE neuron activation
3. Replace the model's activation by the SAE reconstructed activations
4. Observe the change in the model's output

Results in practice:

- Coherent concept steering in LLM outputs
- Concept-level guidance in image generation **without retraining**

☞ **Tool to prevent harmful content generation beyond prompt engineering**

Notes

Model auditing

Idea: use unpacked concepts to **inspect what a model has learned**
☞ Before deploying it, or for auditing purposes

Use cases:

- **Bias detection:** Check whether concepts based on protected attributes are encoded
- **Safety monitoring:** Identify neurons encoding harmful or undesirable content
- **Knowledge maps:** Trace which concepts drive the model across layers

Limitations:

- Audit quality depends on the SAE (which is a black-box)
- Not all concepts are recoverable or interpretable
- Absence of a neuron $\neq$ absence of a concept

☞ **Promising direction, but not a silver bullet for AI safety**

Conclusion

What we covered:

- Neural networks encode **concepts as directions** in high-dimensional spaces
- The **superposition hypothesis** explains why neurons alone are not interpretable
- **Sparse autoencoders** offer a tractable way to unpack these superposed concepts
  ☞ At the cost of trading one black box for another
- This machinery can be applied to both **LLMs** (text) and **Stable Diffusion** (images)
- Unpacked concepts enable **concrete downstream applications**
  ☞ e.g., activation steering, concept unlearning, model auditing

A word of caution:

- **Results are mainly empirical**: Theoretical guarantees are largely absent
- **The field is moving fast**: Many open problems remain (Sharkey et al. 2025)

Notes

Ressources

3Blue1Brown's series on Neural Networks
☞ Mathematics behind neural networks, LLMs and image generation models

Neel Nanda's (Deepmind) talk on mechanistic interpretability at IASEIA
☞ A great introduction to mechanistic interpretability

Anthropic's interpretability papers and blog posts
☞ In depth papers and projects on using SAEs to steer/understand LLMs

References i

Ho, J., A. Jain, and P. Abbeel. 2020. "Denoising Diffusion Probabilistic Models." In *Advances in Neural Information Processing Systems*, 33:6840–6851. Curran Associates, Inc. Accessed May 1, 2026. https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html.

Olah, C., N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter. 2020. "Zoom In: An Introduction to Circuits." *Distill* 5, no. 3 (March): 10.23915/distill.00024.001. ISSN: 2476-0757. https://doi.org/10.23915/distill.00024.001. https://distill.pub/2020/circuits/zoom-in.

Radford, A., J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, et al. 2021. "Learning Transferable Visual Models From Natural Language Supervision" [in en]. In *Proceedings of the 38th International Conference on Machine Learning*, 8748–8763. PMLR, July. Accessed May 1, 2026. https://proceedings.mlr.press/v139/radford21a.html.

Sharkey, L., B. Chughtai, J. Batson, J. Lindsey, J. Wu, L. Bushnaq, N. Goldowsky-Dill, et al. 2025. "Open Problems in Mechanistic Interpretability." Survey Certification, *Transactions on Machine Learning Research*, ISSN: 2835-8856. https://openreview.net/forum?id=91H76m9Z94.

Notes

Thank you for your attention!

Any questions?

MAROUANEILIDRISSI.COM

We acknowledge the support of the Canadian Statistical Sciences Institute (CANSSI) and the Natural Sciences and Engineering Research Council of Canada (NSERC)

Nous reconnaissons le soutien de l'Institut Canadien des Sciences Statistiques (INCASS) et du Conseil de Recherches en Sciences Naturelles et en Génie du Canada (CRSNG)

Notes